Convolutional Neural Networks (CNNs) - Lab 2

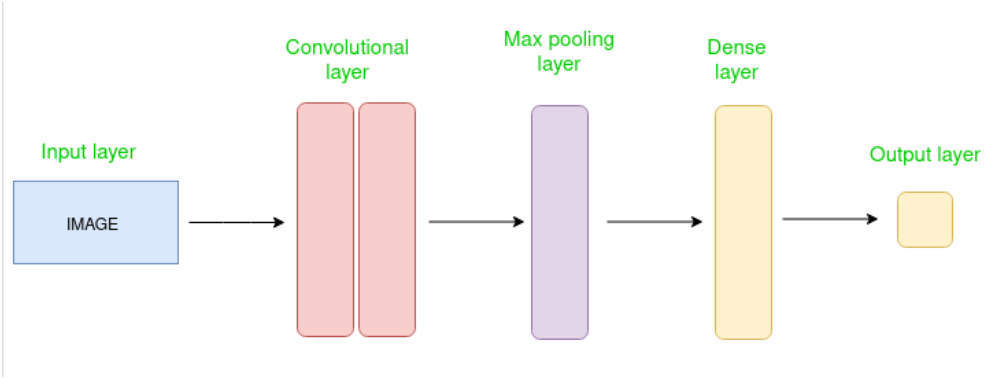
Lab Overview

- Introduction Convolutional Neural Networks (CNNs) process images.
- Identify the role of convolution, activation, pooling, flattening, and fully connected layers.
- Train a simple CNN for multi-class image classification.
- Evaluate the model using accuracy/loss curves, a classification report, and a confusion matrix.

Contents

What is CNN?

A Convolutional Neural Network (CNN) is a neural network architecture designed to learn features from grid-like data (especially images). Unlike a traditional fully connected network that connects every input pixel to every neuron, a CNN uses small learnable filters (kernels) that slide across the input image to extract local patterns such as edges, corners, textures, and then higher-level shapes.



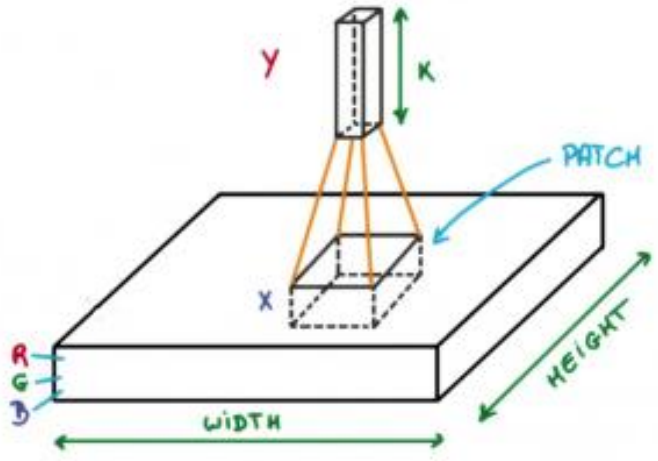
Why CNNs Work Well for Images

- Images have strong spatial structure: neighboring pixels are related.
- CNNs keep this structure by using convolutions instead of flattening the image early.
- Weight sharing: the same filter is applied across the whole image, which reduces parameters and helps generalization.
- Hierarchical feature learning: early layers learn simple patterns (edges), later layers combine them into complex shapes.

The Input Volume (H x W x C)

An RGB image can be seen as a 3D volume:
H = height, W = width, C = channels (3 for RGB: Red, Green, Blue)

CNN layers transform one volume into another. For example, an input of 128 x 128 x 3 can become 128 x 128 x 32 after the first convolution (if we use 32 filters).



Convolution Layer (Feature Extraction)

A convolution layer contains several learnable filters (kernels). Each filter is a small matrix such as 3x3 or 5x5, with the same depth as the input. The filter slides (convolves) across the image. At

each position, it computes a dot product between the filter weights and the corresponding image patch. This produces a 2D activation map (feature map) for that filter.

Key ideas:
- Filters learn automatically during training.
- Each filter becomes specialized (e.g., detecting vertical edges).
- Using multiple filters creates multiple feature maps, increasing the depth of the output volume.

Stride and Padding

- Stride: how far the filter moves each step. Stride=1 moves one pixel at a time; larger strides reduce output size.
- Padding: how borders are handled. 'SAME' padding keeps output spatial size close to the input by adding zeros around edges.

Activation Layer (Non-linearity)

After convolution, an activation function is applied element-wise. The most common is ReLU:
ReLU(x) = max(0, x)

Why it matters:
- Convolution is linear.
- Without non-linearity, stacking layers is still effectively linear.
- ReLU enables complex, non-linear decision boundaries.

Pooling Layer (Down sampling)

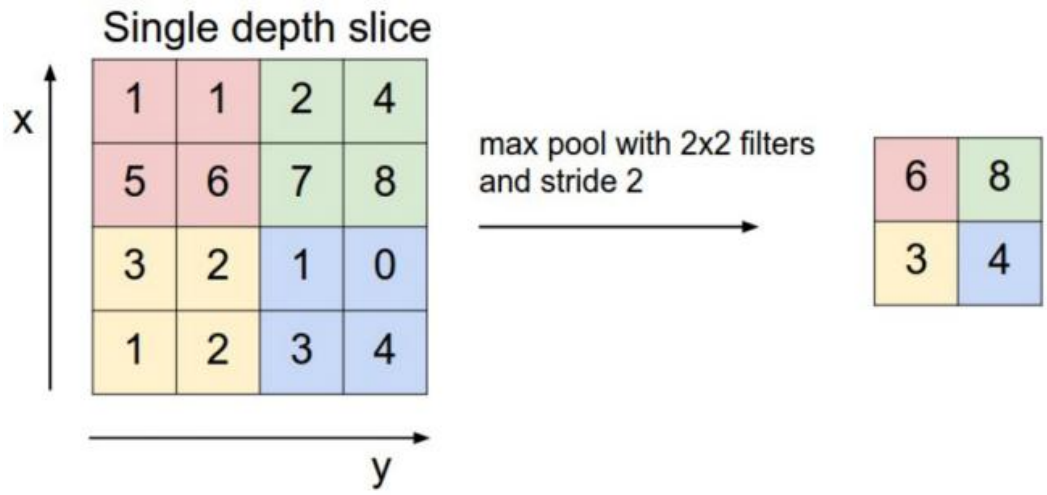
Pooling layers reduce the spatial dimensions (height and width) of feature maps while keeping the most useful information. A common choice is MaxPooling with a 2x2 window and stride 2.

Benefits:
- Reduces computation and memory.
- Adds robustness to small translations.
- Helps reduce overfitting by simplifying representations.

Flattening and Fully Connected Layers

After several convolution + pooling stages, we typically flatten the 3D feature maps into a 1D vector. Fully connected (Dense) layers then combine these learned features to make the final decision.

Output layer:
- For multi-class classification, we often use Softmax.
- Softmax converts scores into probabilities that sum to 1.



Advantages and Disadvantages

Advantages

- Strong at learning visual patterns directly from raw pixels.
- End-to-end learning reduces manual feature engineering.
- Weight sharing reduces parameters compared to fully connected networks.
- Can achieve high accuracy on vision tasks with enough data.

Disadvantages

- Training can be computationally expensive.
- May overfit on small datasets without regularization.
- Often needs lots of labeled data.
- Interpretability is limited (hard to fully explain what each filter learns).

Notebook Code (Explained)

Notes before you run the notebook:
- Some paths in the notebook are Windows-specific (C:\Users\...). Change them to your own dataset location.
- Lines starting with `!` are notebook shell commands.
- If you see an error about a missing library, install it (or run the provided pip cell).
- The last inference cell uses `image.load_img` / `image.img_to_array` and needs this import:
  from tensorflow.keras.preprocessing import image

Code Cell 1: Environment Setup (Install Packages) (Notebook cell index: 1)

```
# (Optional) Install extras if needed (run only if you get ModuleNotFoundError)
!pip install librosa transformers torch nltk scikit-learn matplotlib
```

- (Optional) Install extras if needed (run only if you get ModuleNotFoundError)
- Notebook shell command runs in the system shell (not Python).

Code Cell 2: Imports (Libraries Used) (Notebook cell index: 4)

```
import tensorflow as tf
from tensorflow.keras import layers, models
import kagglehub

# Download latest version
path = kagglehub.dataset_download("pavansanagapati/images-dataset")

print("Path to dataset files:", path)
```

This code cell prepares the environment and downloads the dataset from Kaggle. First, tensorflow as tf is imported so we can build and train the CNN, and layers, models are imported from tensorflow.keras to define the network architecture in a clean way. Then kagglehub is imported, which is a Kaggle-specific tool that can download datasets directly into your local machine or notebook environment. The comment “Download latest version” indicates that the next line will fetch the most recent dataset release, and kagglehub.dataset_download("pavansanagapati/images-dataset") downloads that specific dataset and returns the local folder path where the files were saved. Finally, print("Path to dataset files:", path) displays this location so you can confirm where the dataset exists on your system and use it later when loading images. Keep in mind: the dataset identifier string ("pavansanagapati/images-dataset") is specific to this dataset, and it will change if you choose a different Kaggle dataset.

Code Cell 3: Check Dataset Structure (Notebook cell index: 6)

Code

```
import os
path = r"C:\Users\Lenovo\.cache\kagglehub\datasets\pavansanagapati\images-dataset\versions\1\data"
print(os.listdir(path))
```

This code cell is used to check the dataset directory structure after downloading it. First, the os module is imported because it provides functions for interacting with the operating system, especially files and folders. Next, the path variable is set to a specific local folder on your machine (here, a Windows path inside the KaggleHub cache) that should contain the dataset files—this exact location may differ from one computer to another depending on the username, operating system, and where KaggleHub stores data. Finally, print(os.listdir(path)) displays the contents of that folder (files and subfolders), which helps you confirm that the dataset is there and lets you identify the class folders you'll load later using image dataset from directory.

Code Cell 4: Load Image Dataset from Folder (Notebook cell index: 7)

```
img_size = (128, 128)
batch_size = 32

train_data =
tf.keras.utils.image_dataset_from_directory(r"C:\Users\Lenovo\.cache\kagglehub\datasets\pavansanagapati\images-dataset\versions\1\data",
    image_size=img_size,
    batch_size=batch_size,
    label_mode="categorical"   # for 3 classes
)

# Quick check: what are the class names?
class_names = train_data.class_names
print("Classes:", class_names)
```

In this code cell, you first set two key hyperparameters: `img_size = (128, 128)` defines the target resolution that every image will be resized to (so the CNN receives a consistent input shape), and `batch_size = 32` controls how many images are processed together in each training step (a batch). Next, you load the dataset from the folder using TensorFlow's directory loader, which automatically reads images from subfolders, resizes them to

`image_size=img_size`, groups them into batches using `batch_size=batch_size`, and generates labels based on the folder names. The setting `label_mode="categorical"` means the labels are one-hot encoded, which matches a softmax output layer and the `categorical_crossentropy` loss used later. Finally, the “Quick check” section extracts `class_names` in the exact order TensorFlow detected them and prints them; this is important because that class order must align with the model’s output indices (so class 0 in the output really corresponds to the correct folder/class).

Code Cell 5: Normalize Images (Rescaling) (Notebook cell index: 9)

```
normalization_layer = layers.Rescaling(1./255)
train_data = train_data.map(lambda x, y: (normalization_layer(x), y))
```

This part of the code performs image normalization, which is a standard preprocessing step for CNNs. The first line rescales pixel values from the original range 0–255 down to 0–1, which makes training more numerically stable and helps the optimizer learn more efficiently. The second line uses .map() to apply this normalization step to every batch in the dataset automatically, so each batch of images is scaled correctly before being fed into the model (without changing the original image files on disk).

Code Cell 6: Build CNN Model (Notebook cell index: 11)

```
num_classes = len(class_names)
print("Detected classes:", num_classes, class_names)

model = tf.keras.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    layers.MaxPooling2D(),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes, activation='softmax')
```

```
])
model.summary()
```

In this section, the code first calculates how many categories the model must predict by setting num_classes = len(class_names), then prints the detected number of classes along with their names to confirm that the dataset was read correctly. After that, it builds the CNN using a Sequential model, meaning layers are stacked in order from input to output. The network contains several Conv2D + MaxPooling blocks: each convolution layer learns local visual patterns (like edges, textures, and simple shapes) while the ReLU activation introduces non-linearity, and each max pooling layer downsamples the feature maps to reduce computation and make the model more robust to small shifts in the image. Once enough features are extracted, the Flatten layer converts the 3D feature maps into a 1D vector, which is then passed to a Dense layer to learn higher-level combinations of features. Finally, the output Dense layer uses softmax to produce probabilities across all classes (so the probabilities sum to 1), and model.summary() prints the full architecture with output shapes and parameter counts so you can verify the model structure before training.

Code Cell 7: Compile Model (Notebook cell index: 12)

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

This part of the code compiles the model, meaning it sets up the training configuration before learning starts. It defines the optimizer (how the weights are updated), the loss function (what error the model tries to minimize), and the metrics (what we track during training). Here, Adam is chosen as the optimizer because it adapts the learning rate automatically and is a strong default for CNNs. The loss is categorical cross-entropy, which is the standard choice when your labels are one-hot encoded, and your output layer uses softmax. Finally, metrics=['accuracy'] tell Keras to report accuracy each epoch so you can monitor how often the model predicts the correct class while training progresses.

Code Cell 8: Train Model (Notebook cell index: 13)

```
history = model.fit(
    train_data,
    epochs=10
)
```

This part of the code starts the training process using model.fit(), where the CNN learns by repeatedly seeing batches of images and updating its weights to reduce the loss. The training data is provided through train_data, which is the dataset pipeline containing the images and labels prepared earlier. The parameter epochs=10 means the model will train for 10 full passes over the entire training dataset; after each epoch, the model typically becomes better at recognizing patterns, although too many epochs can lead to overfitting if not monitored. The training results (such as accuracy and loss per epoch) are saved automatically in a History object, which you can later use to plot learning curves and analyze how training progressed over time.

Code Cell 9: Load Dataset with Train/Validation Split (Notebook cell index: 14)

```
img_size = (128, 128)
batch_size = 32

dataset_path = r"C:\Users\Lenovo\.cache\kagglehub\datasets\pavansanagapati\images-
dataset\versions\1\data"

train_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,     # 20% for validation
    subset="training",
    seed=123,
    image_size=img_size,
    batch_size=batch_size,
    label_mode="categorical"
)

val_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=img_size,
```

```
    batch_size=batch_size,
    label_mode="categorical"
)
```

This code cell sets up the main dataset configuration and creates two datasets: one for training and one for validation. First, img_size = (128, 128) defines the target size that all images will be resized to so the model always receives a consistent input shape, and batch_size = 32 determines how many images are processed together in each training step. Then dataset_path is set to the local folder containing the dataset (this is a machine-specific Windows path, so it may differ on other computers).
After that, image_dataset_from_directory() is called twice: the first call loads the training subset and the second loads the validation subset, using validation_split to reserve a portion of the data for validation. The subset argument selects whether we are loading "training" or "validation", while seed=123 ensures the split is reproducible (the same images go to train/validation every run). The parameters image_size=img_size and batch_size=batch_size enforce resizing and batching, and label_mode="categorical" creates one-hot encoded labels, which matches using a softmax output layer with categorical cross-entropy loss later in training.

Code Cell 10: Normalize Images (Rescaling) (Notebook cell index: 15)

```
normalization_layer = layers.Rescaling(1./255)

train_data = train_data.map(lambda x, y: (normalization_layer(x), y))
val_data   = val_data.map(lambda x, y: (normalization_layer(x), y))
```

This section prepares the training and validation data for the CNN by applying normalization in a consistent way. First, the code rescales image pixel values from the usual 0–255 range down to 0–1, which improves numerical stability and typically helps the optimizer learn faster and more reliably. Then, the. map() operation is used to apply this preprocessing step automatically to every batch in the dataset pipeline. Because this mapping is applied to both the training dataset and the validation dataset, it guarantees that the model sees inputs on the same scale during training and evaluation, without modifying the original image files stored on disk.

Code Cell 11: Train Model (Notebook cell index: 16)

```
history = model.fit(
    train_data,
    epochs=10,
    validation_data=val_data
)
```

This code cell trains the CNN using model.fit(), which repeatedly feeds the network batches of images from train_data and updates the model weights to reduce the loss. The parameter epochs=10 means the model will make 10 full passes over the training dataset, allowing it to gradually learn stronger and more generalizable features. By providing validation_data, the model is also evaluated on the validation set at the end of each epoch, producing val_loss and val_accuracy; this is essential for monitoring generalization and detecting overfitting (for example, when training accuracy keeps improving but validation accuracy stops improving or starts decreasing). The training results are stored in a History object, which you later use to plot the learning curves for both training and validation performance.

Code Cell 12: Plot Training Curves (Notebook cell index: 17)

```
import matplotlib.pyplot as plt
# Accuracy
plt.figure()
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='val')
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Loss
plt.figure()
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='val')
plt.title("Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

This code cell visualizes the model's learning progress using Matplotlib. First, matplotlib.pyplot as plt is imported to create plots, then the "Accuracy" section starts by opening a new figure and plotting two curves across epochs: the training accuracy and the validation accuracy stored in the history object returned by model.fit(). The title and axis labels are added to make the graph readable, and plt.legend() is used to clearly distinguish the training curve from the validation curve before displaying the plot. After that, the "Loss" section repeats the same process for loss values: it creates another figure, plots training loss and validation loss over epochs, adds labels and a legend, and then shows the final loss graph. Together, these two plots are essential for diagnosing model behavior—especially spotting overfitting when training improves but validation performance stalls or degrades.

Code Cell 13: Environment Setup (Install Packages) (Notebook cell index: 19)

```
# !pip install scikit-learn

import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
y_true = []
y_pred = []

for images, labels in val_data:
    preds = model.predict(images, verbose=0)

    # one-hot -> class index
    y_true.extend(np.argmax(labels.numpy(), axis=1))
    y_pred.extend(np.argmax(preds, axis=1))

y_true = np.array(y_true)
y_pred = np.array(y_pred)

print(classification_report(
    y_true,
    y_pred,
    target_names=class_names,   # list of class names
    digits=4
))
```

This code cell evaluates the trained model using scikit-learn metrics to get more detailed performance results than accuracy alone. It begins with a comment noting that scikit-learn

may need to be installed in the notebook environment, then imports numpy as np for array handling and imports classification_report and confusion_matrix from sklearn.metrics to compute standard classification scores. Next, two empty lists (y_true and y_pred) are created to store the true labels and the model's predicted labels for all validation samples. The loop for images, labels in val_data: iterates over the validation dataset in batches, and model.predict(images) generates predicted class probabilities for each image. Since both the labels and predictions are in vector form (one-hot labels and softmax probabilities), np.argmax(...) is used to convert them into single class indices by selecting the position of the largest value (i.e., the chosen class). After collecting all batches, the lists are converted into NumPy arrays for easier processing, and classification_report(...) is printed using target_names=class_names so the results appear under the correct class labels. The digits=4 option formats the output to four decimal places, making the precision, recall, and F1-scores clearer and more suitable for an academic report.

Code Cell 14: Evaluate Model (Report + Confusion Matrix) (Notebook cell index: 20)

```
cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix:\n", cm)
```

This part of the code computes the confusion matrix, which summarizes how well the model classified each class by counting predictions versus the true labels. The confusion matrix is typically arranged so that rows represent the true classes and columns represent the predicted classes, meaning the diagonal values show correct predictions while off-diagonal values reveal which classes the model is confusing. Finally, print("Confusion Matrix:\n", cm) prints the matrix to the output so you can quickly inspect the classification errors and identify patterns of misclassification.

Code Cell 15: Single-Image Inference (Notebook cell index: 22)

```
from tensorflow.keras.preprocessing import image
img_path = r"C:\Users\Lenovo\.cache\kagglehub\datasets\pavansanagapati\images-
dataset\versions\1\test\10560.jpg"

# Load image
```

```
img = image.load_img(img_path, target_size=img_size)

# Convert to array & normalize
img_array = image.img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)

# Predict
prediction = model.predict(img_array, verbose=0)
class_index = np.argmax(prediction)

# Print probabilities nicely
print("Prediction probabilities:")
for i, prob in enumerate(prediction[0]):
    print(f"{class_names[i]}: {prob:.4f}")

print("\nPredicted class:", class_names[class_index])

# Show image with prediction
plt.imshow(img)
plt.title(f"Predicted: {class_names[class_index]}")
plt.axis("off")
plt.show()
```

This code cell demonstrates single-image inference, meaning you use the trained CNN to predict the class of one new image. First, from tensorflow.keras.preprocessing import image is imported to access helper functions for loading and converting images, and img_path is set to the location of the test image on your machine (this Windows path is device-specific and may need to be changed). The image is then loaded and resized to the same input size used during training with image.load_img(img_path, target_size=img_size), ensuring the model receives the correct dimensions. Next, the image is converted into a numeric array and normalized to 0–1 by dividing by 255, and np.expand_dims(..., axis=0) adds a batch dimension so the input shape becomes (1, H, W, C)—because Keras models always expect a batch, even if it contains only one image. The model then predicts class probabilities using model.predict, and np.argmax selects the class index with the highest probability. To make the output easy to interpret, the code prints the probability for each class name in a clean format and then prints the final predicted class. Finally, the image is displayed using Matplotlib with a title showing the predicted label and the axes turned off for a cleaner presentation.

Assignment Requirements

The assignment requires students to use the provided CNN code in a Jupyter Notebook to train an image classifier on a dataset that contains exactly three classes, then evaluate the model and document the results in an academic report. Students must show a correct data set setup (folder structure and train/validation split), , and include the required outputs: accuracy and loss curves, a confusion matrix, sample prediction images, and evaluation metrics such as the classification report and macro F1-score.

Criterion	What to show
Dataset + Correct Setup	3 classes, correct folder structure, train test shown
Preprocessing	Resize + normalization, explained clearly
CNN Architecture	Explain CNN design + correct 3-class output layer
Training Procedure	epochs, reproducibility (seed)
Required Plots	Train test Accuracy curve, loss curve, confusion matrix, sample predictions
Evaluation Metrics	Classification report + macro F1 + interpretation
Report Quality	Academic structure, clarity, discussion, references